

Presenter Guide — SomNOG9 Go Workshop

Track 3: Software Development | Tuesday 22 September 2026

Before the Room Arrives

- [] Clone the repo: `git clone https://github.com/somnog/learn-go && cd learn-go`
- [] Confirm Go 1.21+: `go version`
- [] Stage 0 passes: `go test -v ./stage0/...`
- [] Terminal and README.md visible on projector
- [] Solutions are gitignored — students cannot see them until you push

Reveal a solution (use when a student gives up):

```
git add -f solutions/stageN/  
git commit -m "reveal stage N solution"  
git push
```

Students then run: `git pull`

Session 1 — 09:00 to 11:00

Stage 0 — Setup (15 min)

Goal: Everyone has Go installed and the repo cloned.

Show on screen:

```
git clone https://github.com/somnog/learn-go  
cd learn-go  
go test -v ./stage0/...
```

Expected output:

```
--- PASS: TestGoVersion  
--- PASS: TestModuleName
```

Say: “Run it yourself. Tell me when you see PASS.”

Wait for everyone. Help anyone with errors — most common issues: Go not in PATH, wrong Go version.

II Check: Everyone shows PASS before moving on.

Stage 1 — Variables & Types (30 min)

Goal: Students write their first Go file. They learn: package declaration, functions, variables, `fmt.Sprintf`.

Type live — show this first:

```
package main

import "fmt"

func ConferenceName() string {
    name := "SomNOG9"
    return name
}

func WelcomeMessage(who string) string {
    return fmt.Sprintf("Welcome to SomNOG9, %s!", who)
}

func main() {
    fmt.Println(ConferenceName())
    fmt.Println(WelcomeMessage("Abdi Hassan"))
}
```

Ask the room:

- "What is the zero value of `int`?" (0)
- "What does `:=` do differently from `var`?" (infers the type)

Send them to code:

"Create `stage1/somnog.go`. Run: `go test -v ./stage1/...`"

|| **Check:** `go test ./stage1/...` shows PASS. Reveal solution if needed.

Stage 2 — Control Flow (35 min)

Goal: Students use `if`, `switch`, and `for` inside functions they already know how to write.

Type live:

```

func TrackName(n int) string {
    switch n {
    case 1:
        return "Network Infrastructure"
    case 2:
        return "System & Services"
    default:
        return "Unknown"
    }
}

func CanRegister(enrolled, capacity int) bool {
    if enrolled < capacity {
        return true
    }
    return false
}

func CountAvailable(enrolled []int, capacity int) int {
    count := 0
    for _, e := range enrolled {
        if e < capacity {
            count++
        }
    }
    return count
}

```

Ask the room:

- “Does Go `switch` need a `break`?” (No)
- “What does `_` mean in `for _, e := range`?” (discard the index)

Note on `Greet`: The test checks that the result contains both a time-of-day word AND `"SomNOG9"`.
 Example: `"Good morning at SomNOG9"`.

Send them to code:

“Create `stage2/control.go`. Run: `go test -v ./stage2/...`”

II **Check:** PASS. Reveal solution if needed.

**Break — 10:00 to 10:15****Session 2 — 10:15 to 12:15****Stage 3 — Slices (40 min)**

Goal: Students use slice literals, `append`, `make`, `range`, and slice expressions.

Type live:

```

tracks := []string{"Network Infrastructure", "System & Services"}
fmt.Println(len(tracks))    // 2
fmt.Println(tracks[0])     // "Network Infrastructure"

tracks = append(tracks, "Software Development")

first2 := tracks[:2]       // slice expression

slots := make([]string, 5) // ["", "", "", "", ""]

for i, t := range tracks {
    fmt.Printf("%d: %s\n", i, t)
}

```

Point out: `First(items []string, n int) []string` — takes `n` as argument and returns a slice, not a single element.

Ask the room:

- “What does `append` return?” (a new slice — you must reassign)
- “What is `make([]string, 5)`?” (a slice of 5 empty strings)

Send them to code:

“Create `stage3/slices.go`. Run: `go test -v ./stage3/...`”

II **Check:** PASS. Reveal solution if needed.

Stage 4 — Maps (40 min)

Goal: Students use `make`, `map` read/write, `comma-ok`, `delete`, and `range`. Every function takes the map as its first argument.

Type live:

```

registry := make(map[string][]string)

// Write
registry["abdi@somnog.so"] = []string{"BGP Routing"}

// Read
sessions := registry["abdi@somnog.so"]

// Comma-ok: check if key exists
sessions, exists := registry["layla@somnog.so"]
if !exists {
    fmt.Println("not registered")
}

// Append a session to an existing attendee
registry["abdi@somnog.so"] = append(registry["abdi@somnog.so"], "Go Workshop")

// Delete
delete(registry, "abdi@somnog.so")

// Count
fmt.Println(len(registry))

```

Emphasise: Maps are reference types — changes inside a function are visible outside. No pointer needed.

Ask the room:

- “What do you get if you read a missing key?” (zero value — `nil` for `[]string`)
- “What happens if you write to a nil map?” (panic — always use `make` first)

Send them to code:

“Create `stage4/registry.go` . Run: `go test -v ./stage4/...` ”

II **Check:** PASS. Reveal solution if needed.

 Lunch — 12:15 to 13:15

Session 3 — 13:15 to 15:15

Stage 5 — Structs & Methods (35 min)

Goal: Students define two nested structs and attach value receiver methods.

Type live:

```

type Speaker struct {
    Name string
    Title string
}

type Session struct {
    ID        int
    Title     string
    Speaker   Speaker
    Capacity  int
    Enrolled  int
}

// Value receiver – s is a copy, original unchanged
func (s Session) IsFull() bool {
    return s.Enrolled >= s.Capacity
}

func (s Session) SpotsLeft() int {
    left := s.Capacity - s.Enrolled
    if left < 0 {
        return 0
    }
    return left
}

// Create and use
s := Session{
    ID:        1,
    Title:     "BGP Routing at Scale",
    Speaker:   Speaker{Name: "Abdi Hassan", Title: "CTO, AfricaNet"},
    Capacity:  50,
    Enrolled:  12,
}
fmt.Println(s.IsFull())    // false
fmt.Println(s.SpotsLeft()) // 38

```

Note on NewSession : Signature is `NewSession(id int, title, track string, capacity int) Session` — four arguments. The `Speaker` field is left at its zero value and set by the caller.

Ask the room:

- “What is a value receiver?” (a copy — the original is not changed)
- “What is the zero value of a struct field?” (zero for its type)

Send them to code:

“Create `stage5/session.go` . Run: `go test -v ./stage5/...` ”

II **Check:** PASS. Reveal solution if needed.

Stage 6 — Pointers, Multiple Returns & Errors (35 min)

Goal: Students pass `*Session` to functions to mutate the original, use multiple return values, and handle errors.

Type live:

```
// Without pointer: only the copy changes
func badEnroll(s Session, email string) {
    s.Registered = append(s.Registered, email) // lost
}

// With pointer: the original changes
func Enroll(s *Session, email string) error {
    if len(s.Registered) >= s.Capacity {
        return errors.New("session is full")
    }
    s.Registered = append(s.Registered, email)
    return nil
}

// Multiple returns: value + error
func FindSession(sessions []*Session, id int) (*Session, error) {
    for _, s := range sessions {
        if s.ID == id {
            return s, nil
        }
    }
    return nil, errors.New("not found")
}

// Caller handles both
s, err := FindSession(all, 42)
if err != nil {
    fmt.Println(err)
    return
}
```

Note on ValidateEmail : Returns just `error` (not `(string, error)`). Return `nil` on success, `errors.New(...)` on failure.

Ask the room:

- “What does `&` do?” (takes the address — gives you a pointer)
- “What does `*` in front of a type mean?” (pointer to that type)
- “What is the convention for the second return value?” (error — nil means success)

Send them to code:

“Create `stage6/functions.go` . Run: `go test -v ./stage6/...` ”

II **Check:** PASS. Reveal solution if needed.

Stage 7 — Interfaces (25 min)

Goal: Students define an interface and write a function that works with any type satisfying it.

Type live:

```

type Describable interface {
    Describe() string
}

type Speaker struct{ Name, Title string }

func (sp Speaker) Describe() string {
    return "Speaker: " + sp.Name + " (" + sp.Title + ")"
}

type Session struct{ ID int; Title string }

func (s Session) Describe() string {
    return fmt.Sprintf("Session %d: %s", s.ID, s.Title)
}

func DescribeAll(items []Describable) []string {
    results := make([]string, len(items))
    for i, item := range items {
        results[i] = item.Describe()
    }
    return results
}

// Mixed slice – both types satisfy Describable
items := []Describable{
    Speaker{Name: "Abdi Hassan", Title: "CTO"},
    Session{ID: 1, Title: "BGP Routing"},
}
fmt.Println(DescribeAll(items))

```

Ask the room:

- “Does Go use `implements` ?” (No — it’s automatic)
- “What happens if a type only has one of the required methods?” (does not satisfy the interface)

Send them to code:

“Create `stage7/interfaces.go`. Run: `go test -v ./stage7/...`”

II **Check:** PASS. Reveal solution if needed.

 Break — 15:15 to 15:30

Session 4 — 15:30 to 17:30

Stage 8 — Goroutines & Channels (35 min)

Goal: Students send notifications and fetch URLs concurrently using goroutines, a buffered channel, and WaitGroup.

Type live — step by step:

```
// Step 1: sequential (slow)
for _, email := range emails {
    fmt.Println("sent:", email)
}

// Step 2: concurrent with goroutines
var wg sync.WaitGroup
for _, email := range emails {
    wg.Add(1)
    go func(e string) {
        defer wg.Done()
        fmt.Println("sent:", e)
    }(email) // ← pass email as argument, do NOT use email directly
}
wg.Wait()

// Step 3: collect results with a buffered channel
ch := make(chan string, len(emails))
for _, email := range emails {
    wg.Add(1)
    go func(e string) {
        defer wg.Done()
        ch <- "sent: " + e
    }(email)
}
wg.Wait()
close(ch)
for r := range ch {
    results = append(results, r)
}
```

Ask the room:

- “Why pass `email` as an argument instead of using it directly in the goroutine?” (loop variable changes — goroutine captures the variable, not the value)
- “Why buffer the channel?” (goroutines can send without blocking)

Send them to code:

“Create `stage8/notify.go`. Run: `go test -v ./stage8/...`”

II **Check:** PASS. Reveal solution if needed.

Stage 9 — HTTP Server (50 min) — Code-Along

Goal: Students complete a working REST API by filling in three TODO blocks.

Introduce the skeleton:

```
cat stage9/server.go
```

Walk through the file: `data types`, `store`, `ResetStore`, `NewRouter`, `handleAll`, `handleOne`, `findByID`.

Show the already-implemented handler (GET /sessions/{id}):

```

case "":
    s, ok := findByID(id)
    if !ok {
        http.Error(w, "session not found", http.StatusNotFound)
        return
    }
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(s)

```

Fill in TODO 1 together — GET /sessions:

```

case http.MethodGet:
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(store)

```

Students fill in TODO 2 — POST /sessions:

```

case http.MethodPost:
    var s Session
    json.NewDecoder(r.Body).Decode(&s)
    s.ID = len(store) + 1
    if s.Attendees == nil {
        s.Attendees = []string{}
    }
    store = append(store, s)
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusCreated)
    json.NewEncoder(w).Encode(s)

```

Students fill in TODO 3 — GET /sessions/{id}/attendees:

```

case "attendees":
    s, ok := findByID(id)
    if !ok {
        http.Error(w, "session not found", http.StatusNotFound)
        return
    }
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(s.Attendees)

```

Run tests:

```

go test -v ./stage9/...

```

Run the server and test live:

```
go run ./stage9/

# In a second terminal:
curl http://localhost:8080/sessions
curl http://localhost:8080/sessions/1
curl -X POST http://localhost:8080/sessions \
  -H "Content-Type: application/json" \
  -d '{"title":"IPv6 at the Edge","speaker":{"name":"Nimo
Abdi","title":"Infrastructure Lead"},"track":"Network Infrastructure","capacity":40}'
curl http://localhost:8080/sessions/3/attendees
```

End of Workshop

Congratulations. Students have built a working Go HTTP API from scratch.

What they covered:

- Package system, modules, `go test`
- Variables, types, functions
- Control flow: `if`, `switch`, `for`
- Slices: `append`, `make`, slice expressions
- Maps: reference types, comma-ok, `delete`
- Structs, nested structs, value receiver methods
- Pointers, multiple returns, errors
- Interfaces: implicit satisfaction, polymorphism
- Goroutines, buffered channels, `WaitGroup`
- HTTP server: routing, JSON encoding/decoding, status codes

Next steps to suggest:

- tour.golang.org — official interactive tour
- go.dev/doc/effective_go — idiomatic Go
- Add a database to the Stage 9 API (use `database/sql` with SQLite)
- Add authentication middleware